Recurrent Neural Networks

Tim Book

GENERAL ASSEMBLY

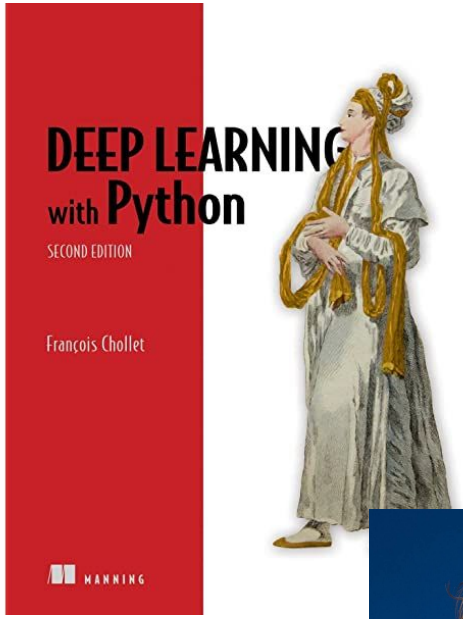
Agenda

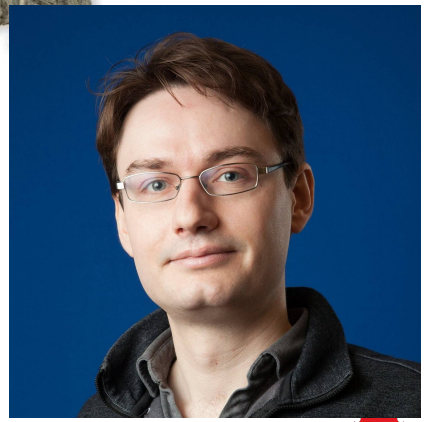
- What is time series data?
- Why do DNNs not handle time series data well?
- What do RNNs do to solve this problem?
- The basic structure of a recurrent layer (the **SimpleRNN**)
- The Long Short Term Memory (**LSTM**) and Gated Recurrent Unit (**GRU**) layers
- Keras notes
- Python! Keras! Yes!

 © 2022 General Assembly

Citation

To give credit where credit is due, many of the examples and images used in this lesson are from **Deep Learning with Python** by **François Chollet**, the author of the Keras library!





What is Time Series Data?

A **time series** is a sequence of data that are **equally spaced** measurements of a phenomenon.

I love finance, so I'll go with an example: **stock prices!**

 © 2022 General Assembly

What is Time Series Data?

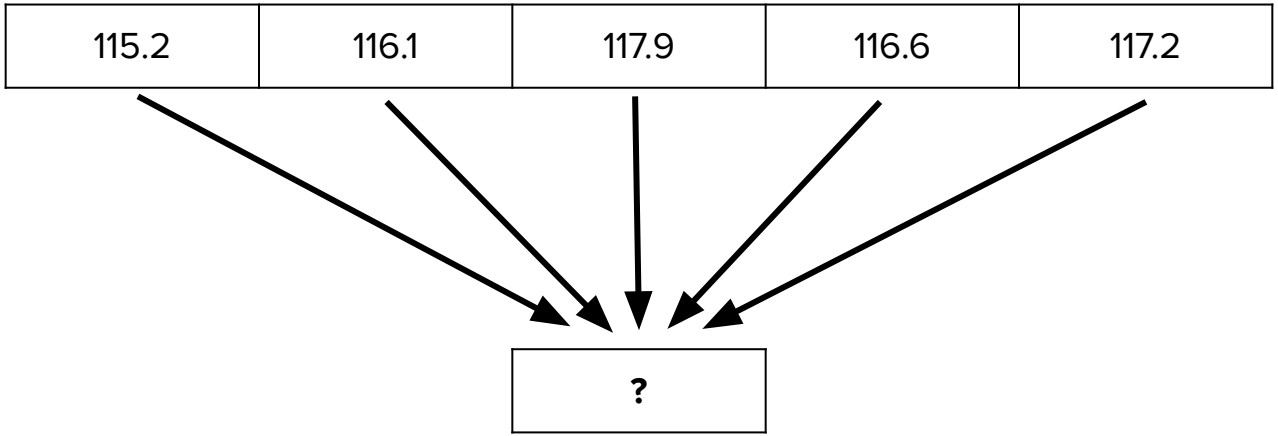
Here's some AAPL stock prices over 5 days:

115.2	116.1	117.9	116.6	117.2	

How can we predict the stock price on day 6?

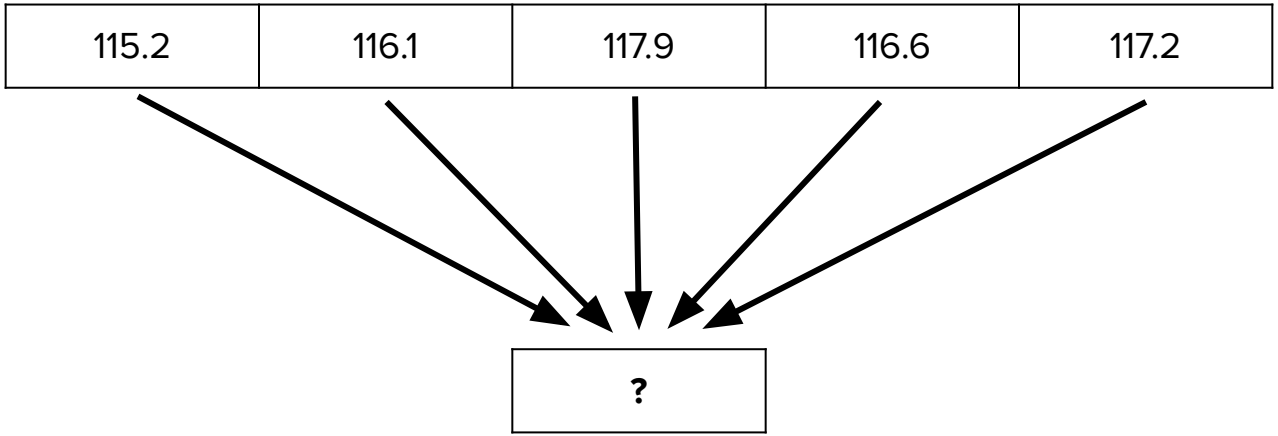
 © 2022 General Assembly

Can we use a DNN? (Dense Neural Network)



 © 2022 General Assembly

Can we use a DNN? (Dense Neural Network)

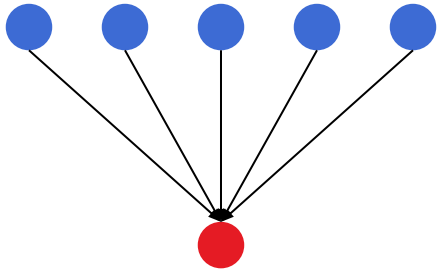


I mean... we ***could***... but what aren't we accounting for when we do this?

 © 2022 General Assembly

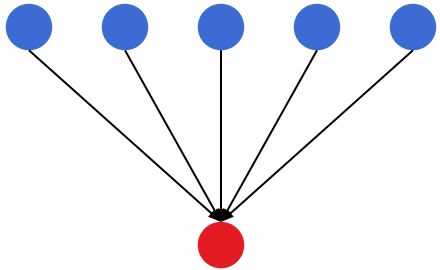
DNNs don’t take sequentiality into account!

Sequence data aren’t logically organized like this…

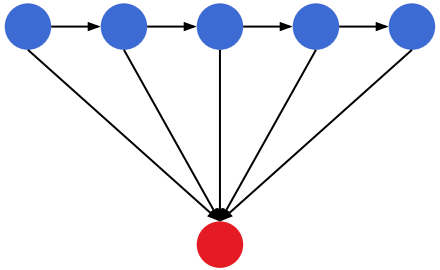


DNNs don’t take sequentiality into account!

Sequence data aren’t logically organized like this…



It’s like this!



RNNs

What do RNNs do?

Here's an ***extremely oversimplified*** for-loop that gives us a gist:

```
state_t = 0
for input_t in input_sequence:
    output_t = f(input_t, state_t)
    state_t = output_t
```

 © 2022 General Assembly

What do RNNs do?

Here's an ***extremely oversimplified*** for-loop that gives us a gist:

```
state_t = 0
for input_t in input_sequence:
    output_t = f(input_t, state_t)
    state_t = output_t
```

An **input sequence** isn't the whole data - it's only one span of input. For example, maybe it's 5 consecutive days of AAPL stock prices, or maybe a sentence in a document.

What do RNNs do?

Here's an ***extremely oversimplified*** for-loop that gives us a gist:

```
state_t = 0
for input_t in input_sequence:
    output_t = f(input_t, state_t)
    state_t = output_t
```

The **output** is a combination of the current state and the current input. Exact details involve matrix algebra.

 © 2022 General Assembly

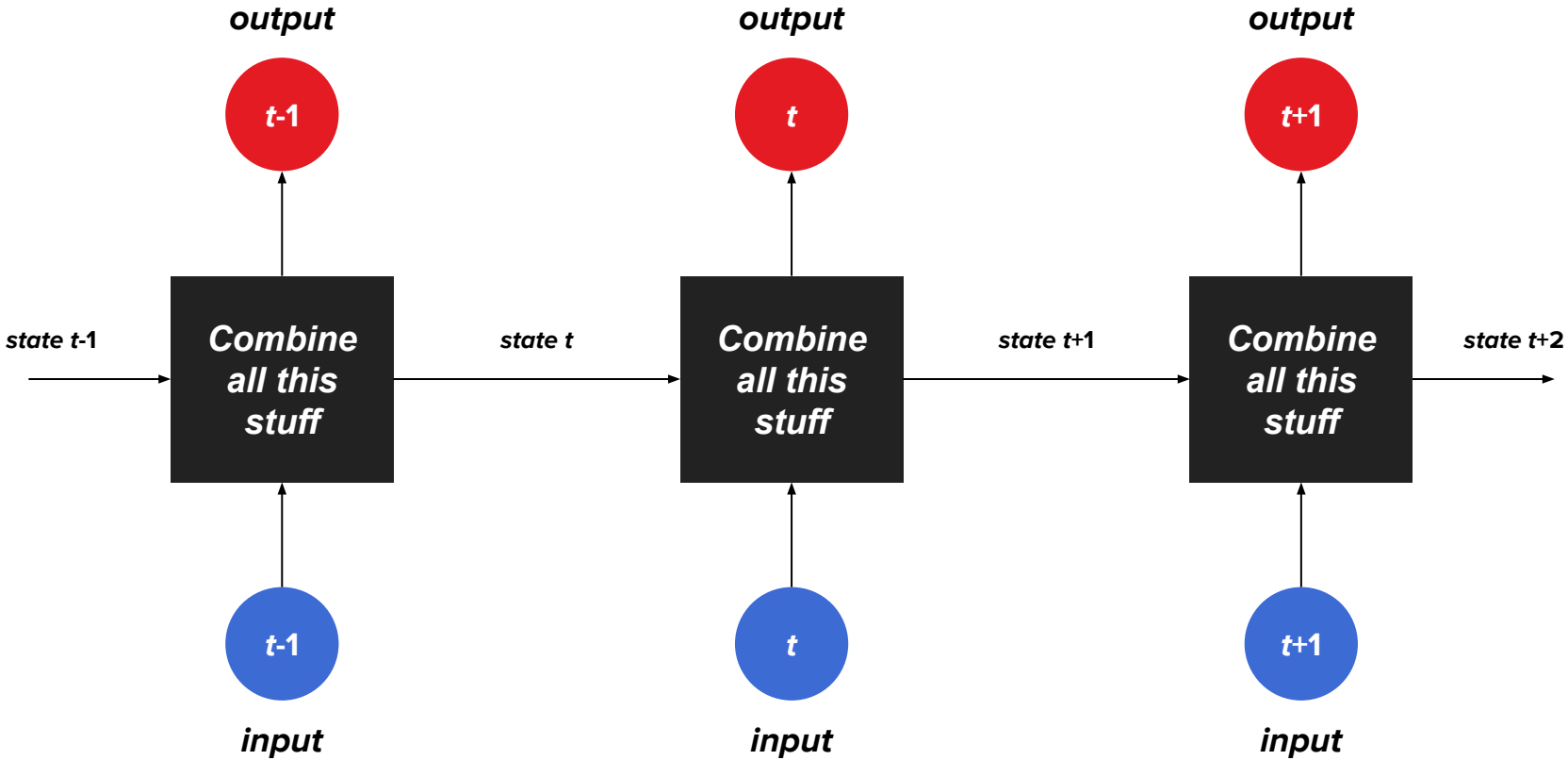
What do RNNs do?

Here's an ***extremely oversimplified*** for-loop that gives us a gist:

```
state_t = 0
for input_t in input_sequence:
    output_t = f(input_t, state_t)
    state_t = output_t
```

Current output is then reassigned to be the current state of the next iteration. This gives the RNN some **"memory"** of the previous input elements in the next input's iteration.

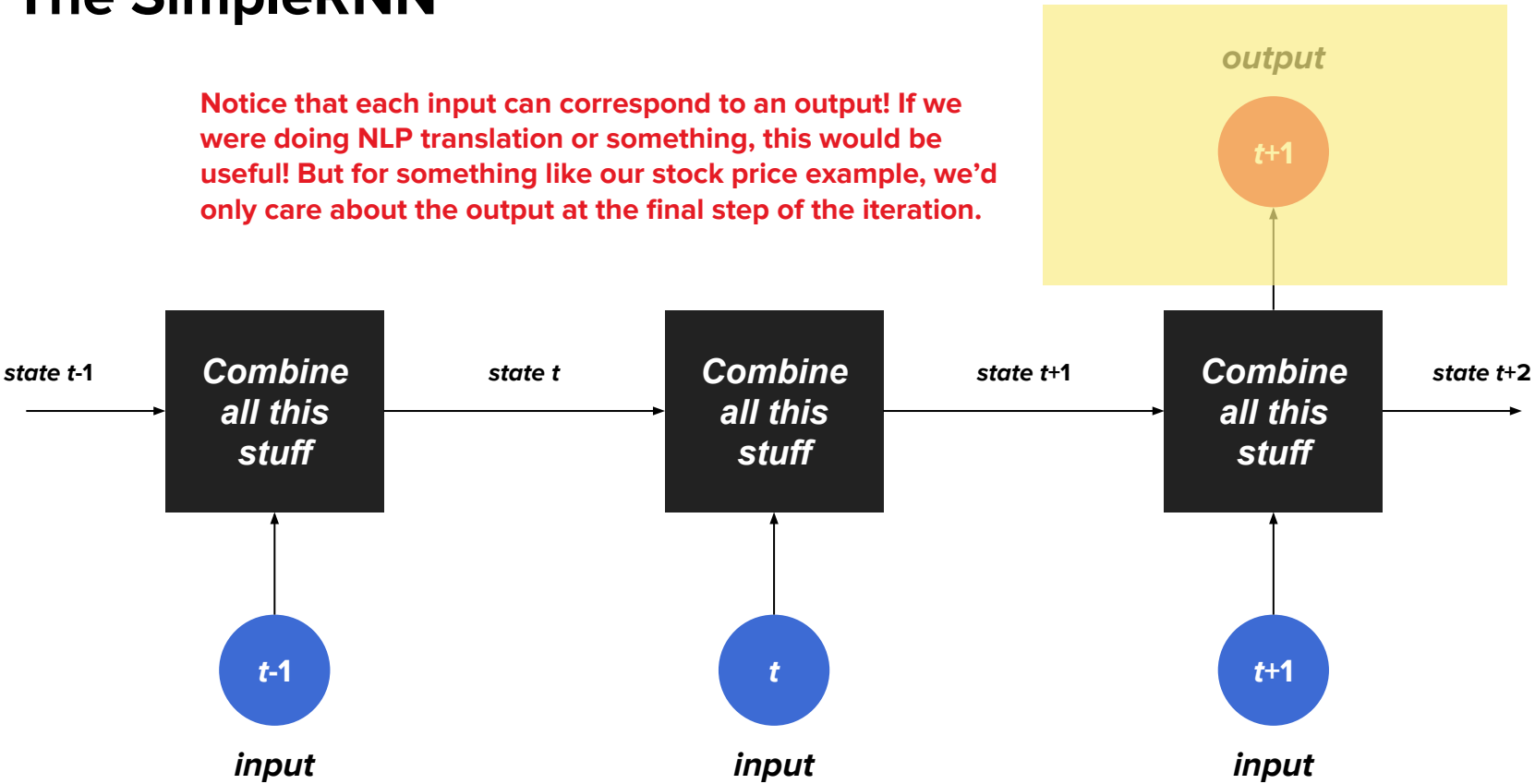
The SimpleRNN



 © 2022 General Assembly

The SimpleRNN

Notice that each input can correspond to an output! If we were doing NLP translation or something, this would be useful! But for something like our stock price example, we'd only care about the output at the final step of the iteration.



Creating the LSTM

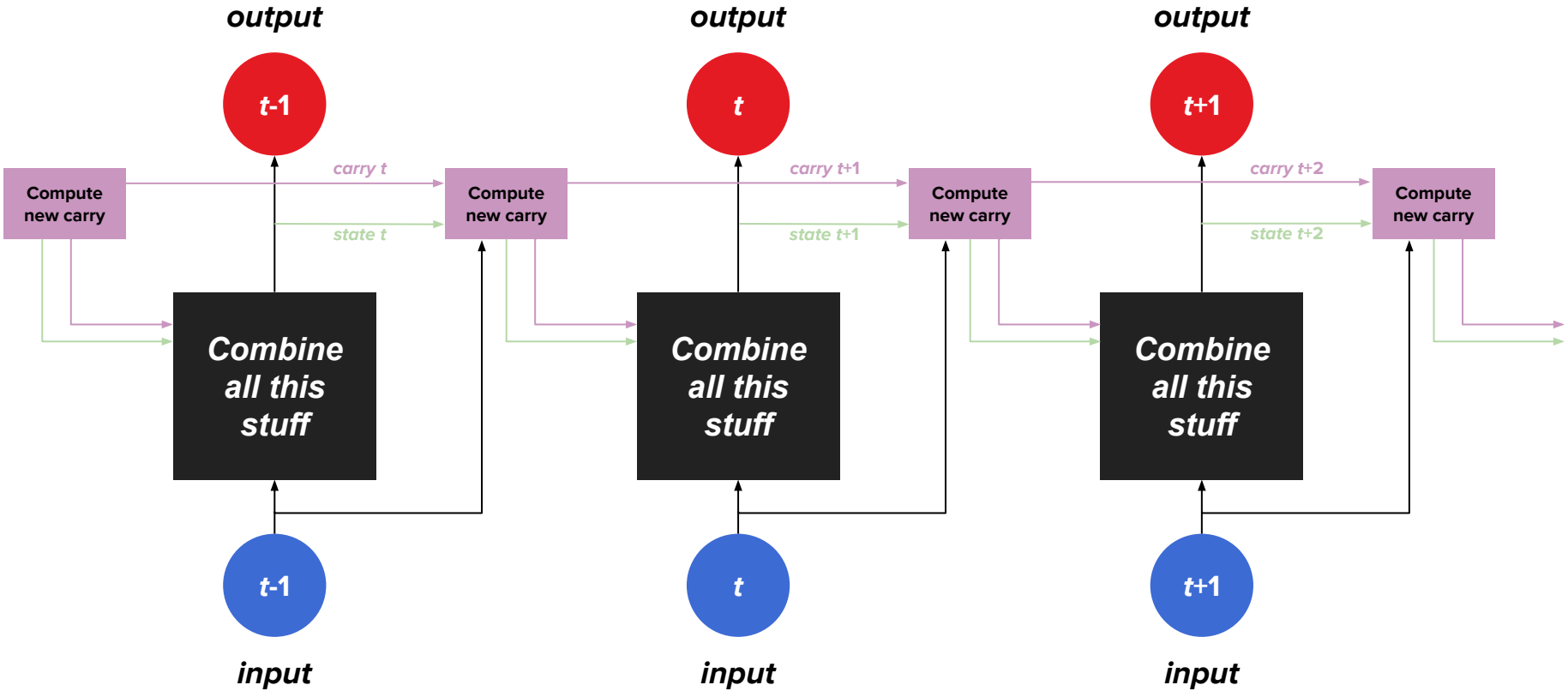
The **Long Short Term Memory (LSTM)** layer is effectively the same as the **Simple RNN** on the previous slide, except for one additional subtlety: a **carry** component that combines three things:

- The current input
- The current state
- The previous carry

Where, the **state** is an updated component that only incorporates the current input and the previous state. Yeah it's a lot going on.

© 2022 General Assembly

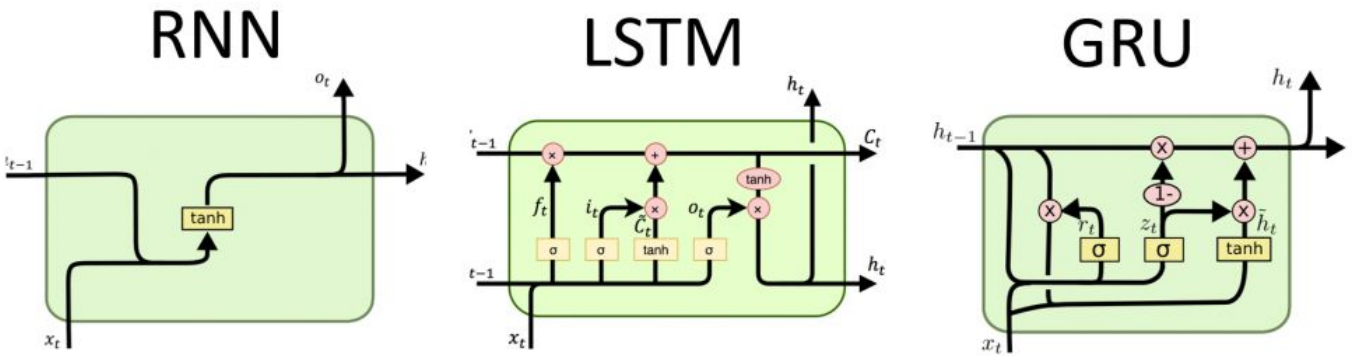
The LSTM



 © 2022 General Assembly

A faster LSTM alternative: GRU

LSTMs can be slow and overpowered. There's a third option: the **Gated Recurrent Unit (GRU)**. It's basically a simplified LSTM that's shown to work almost as well, but is considerably faster.



http://dprogrammer.org/rnn-lstm-gru

 © 2022 General Assembly

Before we begin...

Some Keras Notes



Two important Keras notes:

- How to stack recurrent layers
- How to format data for input

 © 2022 General Assembly

Stacking recurrent layers

By default, recurrent layers in Keras return only the last output in the chain.

If you want to feed your outputs into a new input, that would be equivalent to setting all the outputs in one layer as a sequence of inputs to the next layer.

So, you need to change the default. We can do that by flipping an argument in the layer:

```
LSTM(..., return_sequences=True)
```

Format of Data Input

What **shape** does the input to a recurrent layer have?



Format of Data Input

What **shape** does the input to a recurrent layer have?

Hint: Sequences have a length. You can also incorporate more than one variable in your model.



Format of Data Input

What **shape** does the input to a recurrent layer have? Let's let:

- **`n`** = Size of the full data
- **`seq_len`** = Length of one sequence
- **`n_vars`** = Total number of variables per sequence

So the **`input_shape`** to a layer will be **`(seq_len, n_vars)`**

And the full data will have size **`(n, seq_len, n_vars)`**

 © 2022 General Assembly

Data Processing Example

Let's keep the example of AAPL stock. We'll have a sequence length of **3**.

Low	High	Vol	Close
110	112	1M	111
111	113	1.1M	112
112	114	1.2M	113
113	115	1.1M	114
114	116	1.2M	115

 © 2022 General Assembly

Data Processing Example

Let's keep the example of AAPL stock. We'll have a sequence length of **3**.

Low	High	Vol	Close
110	112	1M	111
111	113	1.1M	112
112	114	1.2M	113
113	115	1.1M	114
114	116	1.2M	115



I want to predict the next day's closing price. How can I set up my **target** vector?

 © 2022 General Assembly

Data Processing Example

Let's keep the example of AAPL stock. We'll have a sequence length of **3**.

Low	High	Vol	Close	target
110	112	1M	111	112
111	113	1.1M	112	113
112	114	1.2M	113	114
113	115	1.1M	114	115
114	116	1.2M	115	116

Data Processing Example

Let's keep the example of AAPL stock. We'll have a sequence length of **3**.

Low	High	Vol	Close	target
110	112	1M	111	112
111	113	1.1M	112	113
112	114	1.2M	113	114
113	115	1.1M	114	115
114	116	1.2M	115	116

Each **sequence** will be of length 3, so our 5-row dataset actually has only 3 sequences in it.

 © 2022 General Assembly

Data Processing Example

Let's keep the example of AAPL stock. We'll have a sequence length of **3**.

Low	High	Vol	Close	target
110	112	1M	111	112
111	113	1.1M	112	113
112	114	1.2M	113	114
113	115	1.1M	114	115
114	116	1.2M	115	116

Each **sequence** will be of length 3, so our 5-row dataset actually has only 3 sequences in it.

 © 2022 General Assembly

Data Processing Example

Let's keep the example of AAPL stock. We'll have a sequence length of **3**.

Low	High	Vol	Close	target
110	112	1M	111	112
111	113	1.1M	112	113
112	114	1.2M	113	114
113	115	1.1M	114	115
114	116	1.2M	115	116

Each **sequence** will be of length 3, so our 5-row dataset actually has only 3 sequences in it.

 © 2022 General Assembly

Data Processing Example

Let's keep the example of AAPL stock. We'll have a sequence length of **3**.

Low	High	Vol	Close	target
110	112	1M	111	112
111	113	1.1M	112	113
112	114	1.2M	113	114
113	115	1.1M	114	115
114	116	1.2M	115	116

Seems like a pain? It is! Thankfully, Keras has a helper function for us to generate this automatically:

`keras.utils.timeseries_dataset_from_array()`

 © 2022 General Assembly

Let's try it out!



 © 2022 General Assembly